

```
func Himitsu() {  
    PasswordManager := 'local-and-secure'  
}
```

```
// Aim of Himitsu
```

```
// Local, secure, lightweight, and portable password manager.
```

```
// User-friendly with a high degree of compatibility.
```

```
KeyFeatures := [3]string {  
    "Portable password storage with securely encrypted  
    XML-based database files",  
    "Web-based GUI using go-webui, allowing for high  
    compatibility",  
    "Secure random password generation, with  
    strength rating based on entropy",  
}
```

technologies_usage.go

// Technologies used

Backend := GOLang

Frontend := HTML + Javascript

UI-Framework := go-webui

Encryption := AES-256-GCM + Argon2id-key-derivation

usage.sh

./himitsu create <file> # Create new database

./himitsu open <file> # Open existing database

./himitsu passwd <file> # Change master password

questions.html

The screenshot shows the Himitsu web application interface. At the top, there's a 'Toggle Lightmode' switch. Below it, a message says 'Database Successfully Loaded.' with a 'Go to Database' button. The main section is titled 'Add New Entry' and contains a form with fields for Title, Username, Email, and Password. The Password field has a 'Generate' button and a strength indicator showing 'KzIDG(z"z"z)N4\$E11'. Below the form is a search bar labeled 'Search by title...'. The bottom section displays three example entries: 'Example Entry', 'Another Example Entry', and 'bookmeter'. Each entry has a 'Copy Password' button. The 'bookmeter' entry shows details like Username (kumi), Email (example@example.com), Password (a complex string), and URL (https://bookmeter.com/). A note at the bottom states: 'Notes: This is a more fleshed out example of a password entry, but does not include any real information except the 'bookmeter' website.'

```

5  "crypto/rand"
6  "io"
7  "math/big"
8  "os"
9
10 "golang.org/x/crypto/argon2"
11 )
12
13 // Function for deriving a secure encryption key from master password
14 func DeriveKey(password string, salt []byte) []byte {
15     key := argon2.IDKey([]byte(password), salt, 1, 64*1024, 4, 32)
16     return key
17 }
18
19 // Function for encrypting database contents
20 func EncryptContents(contents string, key []byte, salt []byte) (encContents []byte, err error) {
21     // Initialize a new aes cipher
22     block, err := aes.NewCipher(key)
23     if err != nil {
24         return nil, err
25     }
26     // Initialize a nonce for the cipher
27     nonce := make([]byte, 12) // Standard GCM nonce size
28     if _, err := io.ReadFull(rand.Reader, nonce); err != nil {
29         return nil, err
30     }
31     // Initialize the gcm cipher
32     gcm, err := cipher.NewGCM(block)
33     if err != nil {
34         return nil, err
35     }
36     // Encrypt the plaintext, appending the nonce
37     contentsBytes := []byte(contents)
38     ciphertext := gcm.Seal(nonce, nonce, contentsBytes, nil)
39
40     return ciphertext, nil
41 }
42
43 // Function for decrypting database contents
44 func DecryptContents(contents []byte, password string) (decContents string, err error) {
45     ciphertext := contents
46
47     // Extract salt from ciphertext

```

security-measures.txt



- AES-256 GCM Encryption.
- Argon2-based key derivation.
- Encryption parameters:
 - ◆ 12 Byte random IV for AES
 - ◆ 16 Byte random salt for key derivation
 - ◆ 32 Byte encryption key derived from salt + master password
- Secure randomization (crypto/rand).
- Password strength validation (go-password-validator).
- Input sanitization for both frontend HTML and backend XML.

// Automated testing w/ Jenkins

```
func Jenkins-testing-pipeline() {  
    Static-analysis := gosec  
    Dependency-scanning := nancy  
}  
JSON_Report := Jenkins-testing-pipeline()
```

manual-testing.txt

- Personal testing during development
- External alpha testing with friends
 - ◆ Helped catch HTML injection vulnerability
 - ◆ Suggested functionality & UI improvements

```
environment {  
    GOPROXY = "https://proxy.golang.org" /usr/lib/go"  
    GOPATH = "${WORKSPACE}/go"  
    PATH = "${GOPROXY}/bin:${GOPATH}/bin:${env.PATH}"  
}  
  
stages {  
    stage('Checkout') {  
        steps {  
            git branch: 'trunk',  
                url: 'https://github.com/k-um-i/himitsu.git'  
        }  
    }  
  
    stage('Setup Go Environment') {  
        steps {  
            sh 'go version'  
            sh 'go mod tidy'  
        }  
    }  
  
    stage('Static Code Analysis (gosec)') {  
        steps {  
            sh 'gosec -no-fail -exclude-dir=pkg -fmt=json -out gosec.json ./...'  
        }  
    }  
  
    stage('Dependency scan (nancy)') {  
        steps {  
            script {  
                try {  
                    sh 'touch nancy.json'  
                    sh 'go list -m all | nancy sleuth --skip-update-check -o json-pretty >  
                } catch (err) {  
                    echo err.getMessage()  
                }  
            }  
        }  
    }  
  
    stage('Archive Results') {  
        steps {  
            archiveArtifacts artifacts: 'gosec.json, nancy.json', fingerprint: true  
        }  
    }  
}
```

```
// Automated testing w/ Jenkins
```

```
func Jenkins-testing-pipeline() {
```

```
    Static-analysis := gosec
```

```
    Dependency-scanning := nancy
```

```
}
```

```
JSON_Report :=
```

```
manual-testing.txt
```

```
→ Personal t
```

```
→ External al
```

- ◆ Helped catch HTML injection vulnerability
- ◆ Suggested functionality & UI improvements

'Jenkins-testing-pipeline()' has crashed



'Jenkins-testing-pipeline()' ran into an unexpected error:
frontend analysis failed:

- unrecognized filetype: 'index.html'
- unrecognized filetype: 'javascript/script.js'

```
environment {  
    GOPATH = "/usr/lib/go"  
    GOPATH = "${WORKSPACE}/go"  
    PATH = "${GOPATH}/bin:${GOPATH}/bin:${env.PATH}"  
}  
  
stages {  
    stage('Checkout') {  
        steps {  
            git branch: 'trunk',  
                url: 'https://github.com/k-un-i/himitsu.git'  
        }  
    }  
}
```

```
fmt=json -out gosec.json ./...
```

```
sleuth --skip-update-check -o json-pretty >
```

```
echo err.getMessage()
```

```
}  
stage('Archive Results') {  
    steps {  
        archiveArtifacts artifacts: 'gosec.json, nancy.json', fingerprint: true  
    }  
}
```

// Future Improvements

security-features.txt

- Add keyfile support for encryption key derivation
- Update Jenkins pipeline to account for frontend

general-features.txt

- Edit and categorize entries
- Password expiration reminders
- Improved UI design

presentation.go

```
<!doctype html>
<html>
  <body>
    <h1>Questions?</h1>

    <div id="Presentator">
      <h2 id="Name">
        [REDACTED]
      </h2>
    </div>
  </body>

  <footer>
    <p>Thank you!</p>
  </footer>
```

questions.html

<https://github.com/k-um-i/himitsu> _ □ ✕

Project Github

- Installation instructions
- Precompiled binaries (win + arch)
- Build instructions

<https://jisho.org/search/himitsu> _ □ ✕

秘密 (himitsu)

Noun, Na-adjective (keiyodoshi), Noun which may take the genitive case particle 'no'

1. secret; secrecy; confidentiality; privacy